

Request Approval Flow Documentation

Overview

This document provides a detailed analysis of the Request Approval flow in DYNAMOS, tracing the complete path from a data analyst's request through the system until a response is returned.

Review of Your Understanding

Your interpretation was **mostly correct**. Here are a few clarifications and corrections:

✅ Correct Interpretations

- API Gateway to Policy Enforcer:** The `requestHandler` sends a `requestApproval` message to `policyEnforcer-in` queue via RabbitMQ.
- Policy Enforcer Agreement Validation:** The `getValidAgreements` function correctly:
 - Loops through requested `dataProviders`
 - Retrieves agreements from etcd at `/policyEnforcer/agreements/{steward}`
 - Validates user relations and archetypes
 - Builds valid/invalid dataprovider lists
- Agreement Redundancy:** You correctly identified that the `agreements` slice is only used to check if agreements exist, which is functionally equivalent to checking `ValidDataproviders`. This appears to be **legacy code** or could be cleaned up.
- Response at Line 84:** Yes, the response is received in the `select` block starting at line 82-84 in `requests.go`.

⚠️ Clarifications/Corrections

1. Routing of `validationResponse`:

- You mentioned the `validationResponse` doesn't contain a `DestinationQueue`. This is **correct**, but not an issue.
- The routing is **hardcoded** in the sidecar's `SendValidationResponse` function at `rabbit_send.go#L174`: it always routes to `"orchestrator-in"`.

2. Orchestrator's Role:

- The orchestrator doesn't just forward the response. It performs **significant processing**:
 - Calls `getAuthorizedProviders()` to check which valid dataproviders are actually online (via etcd `/agents/online/{key}`)
 - Calls `startCompositionRequest()` which:
 - Chooses an archetype based on policy
 - Gets archetype configuration from etcd
 - Sends `CompositionRequest` messages to each authorized provider (data providers and/or compute providers)

- Builds `userTargets` map with DNS endpoints for the client

3. API Gateway Data Flow:

- After receiving `requestApprovalResponse`, the API Gateway calls `sendDataToAuthProviders()` which sends HTTP POST requests **directly to the agents** (not via RabbitMQ).
- The endpoint format is: `http://{url}:{port}/agent/v1/{msgType}/{target}`

Complete Request Approval Flow

Step-by-Step Process

REQUEST APPROVAL FLOW

1. [Client/Data Analyst]

HTTP POST /api/v1/requestApproval
Body: { type, user, dataProviders, dataRequest }



2. [API Gateway – requestHandler()]

- Parses request body
- Creates RequestApproval protobuf
- Creates response channel in requestApprovalMap[user.Id]
- Sets DestinationQueue = "policyEnforcer-in"

RabbitMQ: requestApproval → policyEnforcer-in



3. [Policy Enforcer – handleIncomingMessages()]

- Routes message type "requestApproval"

checkRequestApproval()



4. [Policy Enforcer – getValidAgreements()]

FOR each dataProvider (steward):

- GET etcd: /policyEnforcer/agreements/{steward}
- IF not found → add to invalidDataproviders
- Get user's relation from agreement
- IF user not in relations → add to invalidDataproviders
- Match user's allowedArchetypes with agreement's archetypes
- Match user's allowedComputeProviders with agreement's

computeProviders

- Store in ValidDataproviders and ValidArchetypes

- Set RequestApproved = len(ValidDataproviders) > 0
- Generate Auth token

RabbitMQ: validationResponse → orchestrator-in (HARDCODED)



```

5. [Orchestrator – handleIncomingMessages()]
    • Routes message type "validationResponse"

    handleRequestApproval()
    ▼
6. [Orchestrator – getAuthorizedProviders()]
    FOR each ValidDataProvider:
        • GET etcd: /agents/online/{provider}
        • IF online → add to authorizedProviders

    IF no authorizedProviders → send error response
    ▼
7. [Orchestrator – startCompositionRequest()]
    • chooseArchetype() – selects best archetype
    • GET etcd: /archetypes/{archetype} for config
    • Generate jobName

    IF computeToData archetype:
        • Role = "all"
        • Send CompositionRequest to each provider

    ELSE (dataThroughTtp):
        • Send CompositionRequest(role=dataProvider) to data providers
        • Choose TTP (compute provider)
        • Send CompositionRequest(role=computeProvider) to TTP

    RabbitMQ: compositionRequest → {provider}-in (for each)
    ▼
8. [Orchestrator]
    Build RequestApprovalResponse:
        • authorizedProviders = {name: dns_endpoint}
        • jobId
        • auth tokens
        • user info
        • DestinationQueue = "api-gateway-in"

    RabbitMQ: requestApprovalResponse → api-gateway-in
    ▼
9. [API Gateway – handleIncomingMessages()]
    • Routes message type "requestApprovalResponse"
    • Looks up channel in requestApprovalMap[user.Id]
    • Sends response to channel
    ▼
10. [API Gateway – requestHandler() select block]
    • Receives response from channel
    • Adds requestMetadata to dataRequest
    • Adds trace context

    sendDataToAuthProviders()
    ▼
11. [API Gateway]
    FOR each authorizedProvider:
        HTTP POST http://{dns}:8080/agent/v1/{type}/{target}
        Body: dataRequest with metadata

```

```
▼
12. [Agents]
    | Process data requests...
    | Return responses
    ▼
13. [API Gateway]
    | Aggregate responses
    | Return to client
    ▼
14. [Client/Data Analyst]
    | Receives: { jobId, responses[] }
```

Message Types Summary

Message Type	Source	Destination	Routing
<code>requestApproval</code>	API Gateway	Policy Enforcer	<code>DestinationQueue</code> field (<code>policyEnforcer-in</code>)
<code>validationResponse</code>	Policy Enforcer	Orchestrator	Hardcoded (<code>orchestrator-in</code>)
<code>compositionRequest</code>	Orchestrator	Agents	<code>DestinationQueue</code> field (<code>{agent}-in</code>)
<code>requestApprovalResponse</code>	Orchestrator	API Gateway	<code>RequestMetadata.DestinationQueue</code> (<code>api-gateway-in</code>)

Key Data Structures

RequestApproval (API Gateway → Policy Enforcer)

```
message RequestApproval {
  string type = 1;           // "requestApproval"
  User user = 2;             // {id, user_name}
  repeated string data_providers = 3; // ["VU", "UVA"]
  string destination_queue = 4; // "policyEnforcer-in"
  map<string, bool> options = 5;
}
```

ValidationResponse (Policy Enforcer → Orchestrator)

```
message ValidationResponse {
  string type = 1;           //
  "validationResponse"
  string request_type = 2;   // e.g.,
  "sqlDataRequest"
  map<string, DataProvider> valid_dataproviders = 3; // validated
  providers
```

```

    repeated string invalid_dataproviders = 4;           // rejected providers
    Auth auth = 5;                                     // generated auth
tokens
    User user = 6;
    bool request_approved = 7;
    UserArchetypes valid_archetypes = 8;
    map<string, bool> options = 9;
}

```

RequestApprovalResponse (Orchestrator → API Gateway)

```

message RequestApprovalResponse {
    string type = 1;                                     //
    "requestApprovalResponse"
    User user = 2;
    Auth auth = 3;
    map<string, string> authorized_providers = 4; // {name: dns_endpoint}
    string job_id = 5;
    string error = 6;
    RequestMetadata request_metadata = 7;             // contains
DestinationQueue
}

```

etcd Key Paths

Path	Purpose	Example Value
/policyEnforcer/agreements/{name}	Agreement definitions	See agreements.json
/agents/online/{name}	Online agent status & details	{name, dns, routingKey}
/archetypes/{name}	Archetype configurations	Archetype config JSON

Observations & Potential Improvements

- Redundant agreements slice:** The `agreements` slice in `getValidAgreements` is only used to check length, which is equivalent to checking `ValidDataproviders`. Consider removing.
- Hardcoded routing:** `SendValidationResponse` hardcodes `orchestrator-in`. Consider making this configurable or using a field like other messages.
- Error handling:** Some error paths in `handleRequestApproval` return without sending a response back to the API Gateway, which could cause timeouts.
- Async agent requests:** The `sendDataToAuthProviders` uses goroutines for parallel HTTP requests, which is good for performance.